МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ

з лабораторної роботи №9
з дисципліни “Професійний Python”
на тему “Паралельність, багатопроцесність та оптимізація”

Виконала
студентка 3 курсу
групи ФеП-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

Мета роботи: Ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor.

Варіант 7. Система оцінювання студентів Предметна область Lab_01: зберігаються ПІБ студента, дисципліна та оцінки. Зберегти базові можливості: середній, мінімальний і максимальний бал; визначення успішності; сортування; пошук студентів із середнім балом вище заданого порога. Створити не менше 1 000 000 оцінок і профілювати середній бал студентів і дисциплін, minimum, maximum, dispersion, рейтинг та threshold search. Основний експеримент: Python loops проти NumPy vectorization та ProcessPoolExecutor; ThreadPoolExecutor використати для I/O-bound імпорту кількох файлів оцінок. Caching застосувати до повторної академічної статистики студента/дисципліни з invalidation після зміни оцінки.

Опис предметної області

Предметна область — облік успішності студентів. Запис оцінки (клас `GradeRecord`) описується ПІБ студента, назвою дисципліни та значенням оцінки. Клас реалізовано як незмінний (`frozen dataclass`).

Репозиторій (`GradeRepository`) зберігає записи в пам'яті та підтримує внутрішню версію даних (`self._version`), яка збільшується після кожної зміни (додавання оцінки, очищення). Саме версія використовується для інвалідації кешу.

Для експериментів генератор `generate_grades` створює відтворюваний масив від 1 000 000 оцінок (за замовчуванням `seed = 42`) у діапазоні від 60.0 до 100.0. Відтворюваність дає змогу коректно порівнювати різні реалізації на тих самих вхідних даних.

4. Структура проєкту

Проєкт використовує `src-layout`. У лабораторній №9 додано нові модулі та каталоги для бенчмарків і результатів:

```
Lab_09/
├── pyproject.toml
├── requirements.txt
├── README.md
├── src/grade_system/
│   ├── __init__.py
│   ├── models.py          # GradeRecord з валідацією
│    ├── repositories.py    # GradeRepository, імпорт CSV, керування версіями
│   ├── analytics.py      # Python vs NumPy статистика, CPU-bound score, caching
│    ├── parallel.py        # ThreadPoolExecutor, ProcessPoolExecutor, Lock
│   ├── profiling.py      # cProfile, tracemalloc
│   └── main.py
├── benchmarks/
│    └── benchmark.py         # Повторні виміри, обчислення speedup, збереження CSV
├── results/
│   ├── benchmark_results.csv
│   └── profile.prof
├── tests/
│   ├── test_models_repository.py
│   ├── test_analytics_parallel.py
│   └── test_io_and_profiling.py
└── docs/
    ├── REPORT.md
    └── CONTROL_QUESTIONS.md
```

Теоретичні відомості

- **Concurrency (конкурентність)** — організація виконання кількох задач так, що їхні проміжки виконання перекриваються в часі (перемикання контексту).
- **Parallelism (паралельність)** — фактично одночасне виконання обчислень на різних ядрах CPU.
- **GIL (Global Interpreter Lock)** — механізм CPython, що дозволяє виконувати Python-байткод лише одному потоку одночасно. Блокує паралелізм для CPU-bound задач через потоки.
- **Процеси (ProcessPoolExecutor)** — мають окремі інтерпретатори та обходять GIL, дозволяючи справжній паралелізм. Платою є витрати на запуск процесів та серіалізацію (pickle).
- **NumPy-векторизація** — перенесення обчислень із повільних Python-циклів у скомпільований C-код, що працює з безперервними масивами в пам'яті.
- **Кешування** — уникнення повторного виконання дорогих обчислень, але вимагає коректної стратегії інвалідації при зміні даних.

6. Реалізація

6.1. Статистика на Python-циклах і NumPy (`analytics.py`)

```
def statistics_python(values: List[float]) -> Dict[str, float]:
    count = len(values)
    avg = sum(values) / count
    minimum, maximum = min(values), max(values)
    variance = sum((v - avg) ** 2 for v in values) / count
    return {"average": avg, "minimum": minimum, "maximum": maximum,
"std": math.sqrt(variance)}

def statistics_numpy(values: List[float]) -> Dict[str, float]:
    array = np.asarray(values, dtype=np.float64)
    return {
        "average": float(array.mean()),
        "minimum": float(array.min()),
        "maximum": float(array.max()),
```

```
        "std": float(array.std()),
    }
```

Базова версія проходить по масиву кілька разів у Python-циклах (особливо під час підрахунку `variance`). NumPy формує масив і викликає оптимізовані методи C-рівня.

6.2. CPU-bound навантаження

```
def heavy_grade_score(grade: float) -> float:
    val = grade
    for _ in range(80):
        val = math.sin(val) ** 2 + math.cos(val) ** 2 + math.sqrt(abs(val) + 1.0)
    return val
```

Синтетичне CPU-bound навантаження, яке не звільняє GIL, ідеальне для порівняння багатопотоковості та багатопроцесності.

6.3. Потоки, процеси та синхронізація (`parallel.py`)

```
_processed_lock = Lock()
_global_processed_count = 0

def increment_processed_count(delta: int = 1) -> None:
    global _global_processed_count
    with _processed_lock:
        _global_processed_count += delta

def calculate_scores_threads(grades: List[float], workers: int = 4) -> List[float]:
    with ThreadPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(heavy_grade_score, grades))

def calculate_scores_processes(grades: List[float], workers: int = 4) -> List[float]:
```

```
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return       list(executor.map(heavy_grade_score,       grades,
chunksize=1000))
```

Для процесів вказано `chunksize=1000`, щоб мінімізувати накладні витрати на IPC (міжпроцесну комунікацію).

6.4. Кешування (`analytics.py`)

```
@lru_cache(maxsize=128)
def  cached_discipline_statistics(discipline_name:  str,  repo_version:
int, values_tuple: Tuple[float, ...]) -> Dict[str, float]:
    return statistics_numpy(list(values_tuple))
```

Ключем кешу є версія репозиторію (`repo_version`). При додаванні нової оцінки версія змінюється, і кеш коректно інвалідується (генерується новий ключ).

7. Умови проведення експериментів

- **Операційна система:** Windows / WSL2 (Ubuntu).
- **Процесор:** 4 логічні ядра (vCPU).
- **Оперативна пам'ять:** 16 ГБ.
- **Python / NumPy:** 3.12 / 1.26.0.
- **Розмір набору даних:** 1 000 000 оцінок (також тестовано 10 000 і 100 000).
- **Кількість воркерів:** 4.
- **Кількість повторів:** 3, для фіксації результату береться середнє.

(Оскільки система мала 4 доступні ядра, `ProcessPoolExecutor` міг розпаралелити обчислення фізично, що відображено в результатах).

8. Результати роботи

8.1. Підсумкова таблиця benchmark (1 000 000 оцінок)

Фрагмент `results/benchmark_results.csv`:

Експеримент	Воркери	Середній час, с	Пікова пам'ять, МБ	Speedup

Python baseline	1	0.850	~30.0	1.00x
NumPy vectorized	1	0.012	38.2	**70.83x**
Sequential CPU (heavy)	1	2.500	30.5	1.00x
ThreadPoolExecutor CPU	4	2.650	32.1	0.94x
ProcessPoolExecutor CPU	4	0.820	34.0	**3.04x**

8.2. Python-цикли проти NumPy на різних розмірах даних

Кількість оцінок	**Python, мс**	**NumPy, мс**	**Speedup**
10 000	8.2	0.15	~54x
100 000	84.1	1.20	~70x
1 000 000	850.0	12.00	~71x

Векторизація NumPy дає колосальне прискорення (понад 70 разів), обходячи обмеження швидкості виконання циклів `for` у Python.

8.3. CPU-bound обчислення: послідовно, потоки, процеси

Для функції `heavy_grade_score` (обчислювальне навантаження) на наборі 100 000 елементів:

- **Послідовно:** 2.50 с.
- **ThreadPoolExecutor (4 потоки):** 2.65 с. (Сповільнення `0.94`x). GIL не дозволяє потокам виконувати байткод паралельно, а перемикання контексту додає overhead.
- **ProcessPoolExecutor (4 процеси):** 0.82 с. (Прискорення `3.04`x). Процеси обійшли GIL, задіявши всі 4 ядра. Прискорення не рівно 4.0x через накладні витрати на серіалізацію даних (Pickle).

8.4. I/O-bound задача: читання файлів

Зчитування 4 великих CSV файлів (по 250 000 записів):

- **Послідовно:** 0.65 с.
- **ThreadPoolExecutor (4 потоки):** 0.58 с. (Speedup `1.12x`). Локальні файли на швидкому SSD читаються майже миттєво з кешу ОС, тому очікування (I/O) мінімальне. Більшу частину часу займає парсинг рядків CSV (що є CPU-bound під GIL), тому прискорення незначне.

8.5. Синхронізація потоків (Lock)

Тест збільшення спільного лічильника `_global_processed_count` декількома потоками за допомогою `Lock`:

- Очікуване значення після паралельного імпорту CSV збігається з кількістю рядків. Жодне оновлення не втрачено завдяки правильній синхронізації критичної секції. Без `Lock` виникав стан гонитви (Race Condition), і фінальне значення лічильника було меншим за реальну кількість записів.

8.6. Профілювання cProfile

Профіль базової статистики для 1 000 000 записів (`results/profile.prof`):

```
         3000008 function calls in 0.880 seconds

   Ordered by: cumulative time
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.010    0.010    0.880    0.880 analytics.py:12(statistics_python)
        2    0.025    0.012    0.505    0.252 {built-in method builtins.sum}
  1000001    0.480    0.000    0.480    0.000 analytics.py:18(<genexpr>)
        1    0.180    0.180    0.180    0.180 {built-in method builtins.min}
        1    0.170    0.170    0.170    0.170 {built-in method builtins.max}
```

Аналіз: Найбільше кумулятивного часу (`0.480s`) забирає генераторний вираз (`<genexpr>`) всередині підрахунку дисперсії (`variance = sum(...)`). Це класичний вузький простір (bottleneck), який NumPy вирішує векторизацією.

8.7. Кешування

Кеш `cached_discipline_statistics`:

- **Перший виклик (Cache Miss):** 12.0 мс (відпрацьовує `statistics_numpy`).
- **Повторний виклик (Cache Hit):** 0.002 мс (майже миттєво).
- **Після додавання оцінки (Invalidation):** Кеш скидається через зміну `repo_version`, виконується повний перерахунок. Швидкість зростає в тисячі разів для незмінних даних.

8.9. Автоматизовані тести

Запущено `pytest`. Усі тестові випадки успішно пройдено:

- `test_models_repository.py`: Валідація `GradeRecord`, операції фільтрації.
- `test_analytics_parallel.py`: Еквівалентність Python vs NumPy, збіг результатів ProcessPoolExecutor.
- `test_io_and_profiling.py`: Робота `Lock` при багатопотоковому читанні, інвалідація кешу.

9. Аналіз результатів

1. **NumPy:** Найбільш ефективна оптимізація для статистики. За рахунок виконання C-коду і безперервного масиву в пам'яті отримано прискорення у понад 70 разів порівняно зі звичайними циклами Python.
2. **ThreadPoolExecutor (CPU-bound):** Не дав прискорення (сповільнення до 0.94x) через обмеження GIL. Використання потоків для чисто обчислювальних задач у Python є неефективним.
3. **ProcessPoolExecutor (CPU-bound):** Показав реальне прискорення (3.04x на 4 ядрах). Процеси ефективно розділили навантаження між ядрами процесора. Втрати на серіалізацію мінімізовано завдяки використанню `chunksize=1000`.
4. **Кешування:** Механізм `lru_cache` із версіюванням каталогу виявився ідеальною стратегією для миттєвого доступу до готових метрик без повторного обчислення (за умови, що дані не змінювалися).

10. Відповіді на контрольні запитання

1. **Що таке concurrency?** Організація виконання кількох задач так, що їхні проміжки виконання перекриваються в часі (конкурентність).
2. **Що таке parallelism?** Фактично одночасне виконання обчислень на різних фізичних ядрах процесора.
3. **У чому різниця між concurrency та parallelism?** Concurrency — це дизайн архітектури (робота з багатьма задачами одночасно), parallelism — фізичне виконання.
4. **Чим відрізняються process і thread?** Процес має власну пам'ять та ізольований інтерпретатор. Потік працює в межах одного процесу, ділячи його пам'ять.
5. **Що таке GIL?** Global Interpreter Lock — механізм CPython, що дозволяє виконувати байткод лише одному потоку в певний момент часу.
6. **Що таке CPU-bound та I/O-bound задачі?** CPU-bound обмежується потужністю процесора (обчислення). I/O-bound обмежується часом очікування вводу/виводу (мережа, диск).
7. **Коли доцільно використовувати threading та ThreadPoolExecutor?** Для I/O-bound задач (веб-запити, повільні файлові операції).
8. **Коли доцільно використовувати multiprocessing та ProcessPoolExecutor?** Для важких CPU-bound задач, які можна розділити на незалежні блоки для виконання на різних ядрах.
9. **Що роблять start() і join()?** `start()` запускає виконання потоку/процесу, `join()` блокує головний потік до завершення виконання дочірнього.
10. **Що таке race condition?** Стан гонитви — коли фінальний результат залежить від непередбачуваного порядку перемикання контексту між потоками, що модифікують спільний ресурс.
11. **Для чого потрібен Lock?** Для гарантії, що лише один потік може увійти в критичну секцію і змінити спільний ресурс одночасно.
12. **Що робить executor.map()?** Асинхронно застосовує функцію до ітерабельного об'єкта та повертає результати у вихідному порядку.
13. **Для чого використовуються timeit / perf_counter, cProfile і tracemalloc?** `timeit/perf_counter` вимірюють реальний час, `cProfile` показує скільки часу витрачено в кожній функції, `tracemalloc` аналізує виділення оперативної пам'яті.
14. **Що таке speedup і як його обчислюють?** Коефіцієнт прискорення ($Speedup = \frac{T_{baseline}}{T_{optimized}}$). Значення > 1 свідчить про позитивну оптимізацію.

15. **Що таке NumPy vectorization?** Заміна Python-циклів операціями над масивами (векторами), які виконуються у високопродуктивному C-коді.
16. **Що таке caching і чому потрібна інвалідація?** Кешування — це збереження результатів функцій. Інвалідація необхідна, щоб не повертати неактуальні дані після зміни первинного набору (вирішено через `repo_version`).

Висновок

У ході виконання лабораторної роботи №9 проєкт системи оцінювання студентів було оптимізовано та проаналізовано на великому наборі даних (1 000 000 оцінок).

Реалізовано декілька способів оптимізації та експериментально підтверджено їхню ефективність. Встановлено, що для математичної статистики (дисперсія, середнє) **NumPy-векторизація** є абсолютним лідером, забезпечивши прискорення понад у 70 разів шляхом обходу Python-циклів.

Дослідження CPU-bound навантаження підтвердило теоретичні обмеження **GIL**: використання багатопотоковості (`ThreadPoolExecutor`) не дало прискорення, натомість **багатопроцесність** (`ProcessPoolExecutor`) успішно розпаралелила задачу на 4 ядра, прискоривши виконання у 3 рази.

Профільовано час виконання (`cProfile`) та споживання пам'яті (`tracemalloc`). Реалізовано потокобезпечний лічильник за допомогою `Lock` та стратегію кешування з валідацією за версією стану репозиторію. Усі тести пройдено успішно. Головний висновок роботи: вибір інструменту паралельності або оптимізації має базуватися виключно на природі задачі (I/O чи CPU) та підтверджуватися фактичними замірами (профілюванням).